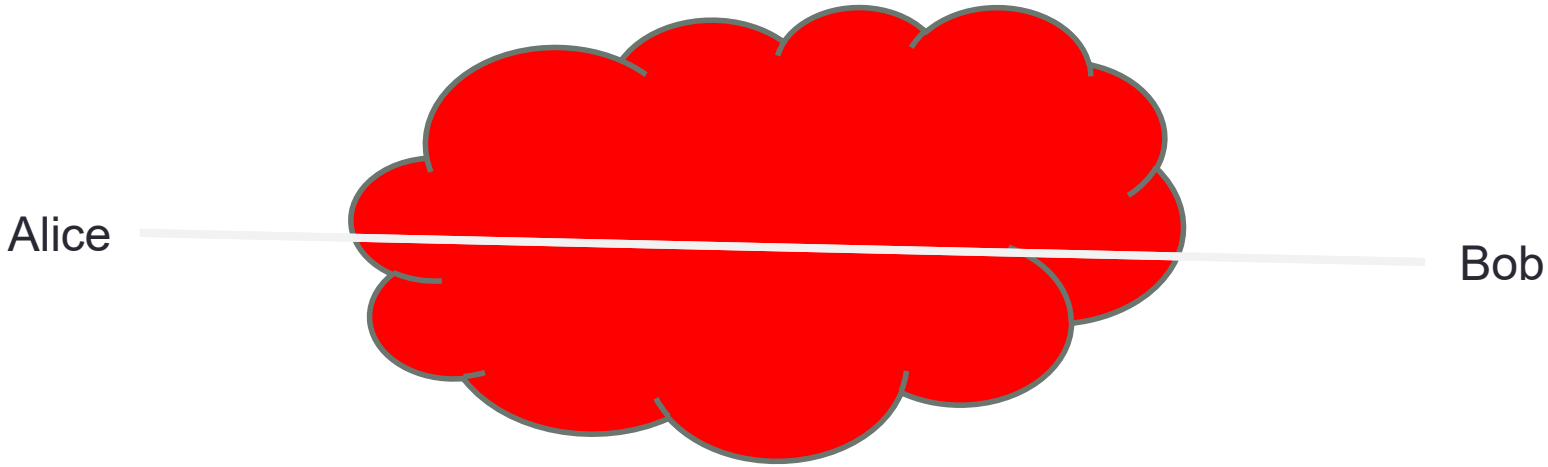
NETWORK SECURITY PRACTICES – ATTACK AND DEFENSE

End-to-End Encrypted Link

Network Security Approaches



Network Security Protocols

- SSL/TLS
  - Secure sockets layer / Transport layer security
  - Used mainly to secure Web traffic
- SSH
  - Secure Shell
  - Remote login
- IPsec
  - IP-level security suite

SSL

- Mid ‘90s introduced concerns over credit card transactions over the Internet
- SSL designed to respond to thse concerns, develop e-commerce
- Initially designed by Netscape, moved to IETF standard later

SSL model

- A client and a server
- Implements a socket interface
  - Any socket-based application can be made to run on top of SSL
- Protect against:
  - Eavesdroppers
  - MITM attacks
- Server has X.509 certificate
  - Client may have a certificate, too

SSL certificates (server certificate)

SSL certificates (CA certificates)

憑證管理員

您的憑證 | 人員 | 伺服器 | 憑證機構 | 其他

您的憑證可識別下列憑證機構:

憑證名稱	安全裝置
VeriSign, Inc.	軟體安全裝置
VeriSign Class 3 Public Primary Certification Authority - G5	Builtin Object Token
VeriSign Universal Root Certification Authority	Builtin Object Token
Verisign Class 3 Public Primary Certification Authority	Builtin Object Token
Verisign Class 1 Public Primary Certification Authority - G2	Builtin Object Token
Verisign Class 2 Public Primary Certification Authority - G2	Builtin Object Token
Verisign Class 3 Public Primary Certification Authority - G2	Builtin Object Token
VeriSign Class 1 Public Primary Certification Authority - G3	Builtin Object Token
VeriSign Class 2 Public Primary Certification Authority - G3	Builtin Object Token
VeriSign Class 3 Public Primary Certification Authority - G3	Builtin Object Token
VeriSign Class 4 Public Primary Certification Authority - G3	Builtin Object Token
VeriSign Class 3 Public Primary Certification Authority - G4	Builtin Object Token
Verisign Class 1 Public Primary Certification Authority	Builtin Object Token
Verisign Class 3 Public Primary Certification Authority	Builtin Object Token
VeriSign Class 3 International Server CA - G3	軟體安全裝置
VeriSign Class 3 Extended Validation SSL SGC CA	軟體安全裝置
VeriSign Class 3 Secure Server CA - G2	軟體安全裝置
VeriSign Class 3 Secure Server CA - G3	軟體安全裝置

檢視 (V)... | 編輯信任 (E)... | 匯入 (M)... | 匯出 (X)... | 刪除或取消信任 (D)...

確定

SSL certificates (client certificate)

SSL history

- SSLv2 1994
- SSLv3 1996
  - Fixed security problems
- TLS v1.0 1999
- TLS v1.1 2006

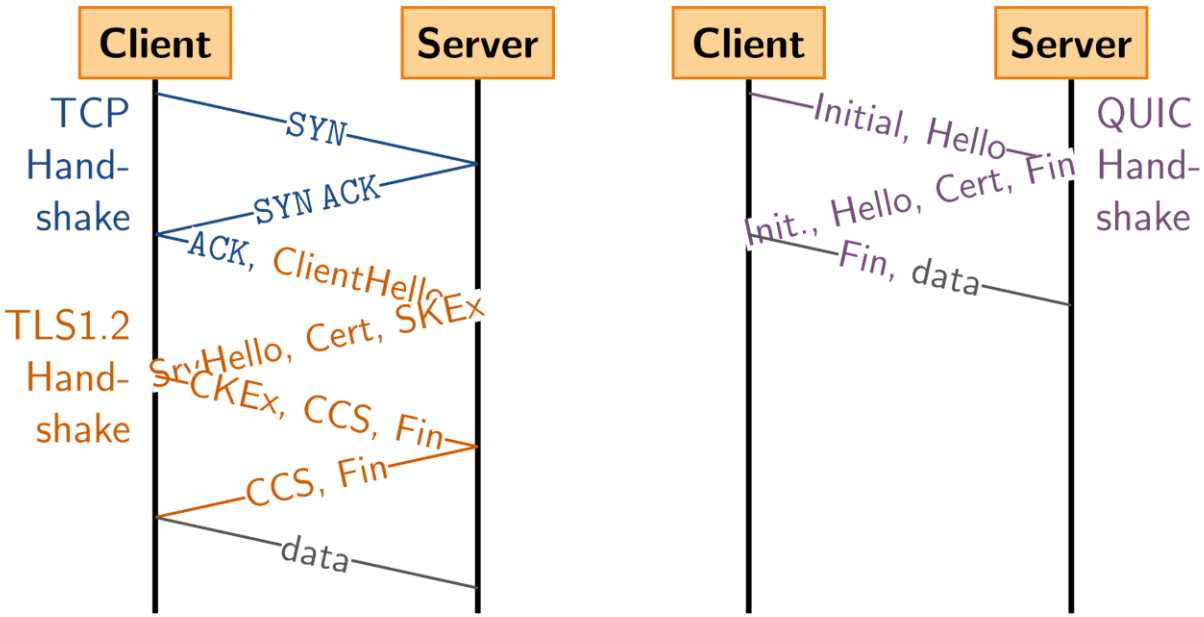
SSL key lengths

- Earlier versions used 128-bit keys for export reasons
- Later versions switched to 256-bit keys
- Rollback attack:
  - MITM

```
Handshake Protocol: Server Hello
    Handshake Type: Server Hello (2)
    Length: 151
    Version: TLS 1.2 (0x0303)
    Random: 42360be02c58e5395001ac909c2d6d6a3da6e51f4c5eeaf79012396b3e9da954
    Session ID Length: 32
    Session ID: ed563c93aaa1ef101e09bf721e1109f528f6948099eef354629888ae4195d69b
    Cipher Suite: TLS_AES_256_GCM_SHA384 (0x1302)
    Compression Method: null (0)
    Extensions Length: 79
    Extension: supported_versions (len=2) TLS 1.3
    Extension: key_share (len=69) secp256r1
    [JA3S Fullstring: 771,4866,43-51]
    [JA3S: 15af977ce25de452b96affa2addb1036]
TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
  Opaque Type: Application Data (23)
  Version: TLS 1.2 (0x0303)
  Length: 36
  Encrypted Application Data: 2861d6c62eb7e516ab3e296e36b85a044aca98cb9a0291e77951d043c8115c7991b7773e
  [Application Data Protocol: Hypertext Transfer Protocol]
TLS segment data (1259 bytes)
```

SSL sequence

- Negotiate parameters
- Key exchange
- Authentication
- Session



SSL negotiation

- Choice of cipher suites, key exchange algorithms, protocol versions
- Rollback attack: MITM chooses least secure parameters

*乙太網路 2

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.addr==140.116.248.21

No.	Time	Source	Destination	Protoco	Lengt	Info
282	4.268645800	192.168.0.100	140.116.248.21	TCP	66	64541 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
283	4.268762200	192.168.0.100	140.116.248.21	TCP	66	53199 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
297	4.274668600	140.116.248.21	192.168.0.100	TCP	60	80 → 53199 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
298	4.274759700	192.168.0.100	140.116.248.21	TCP	54	53199 → 80 [ACK] Seq=1 Ack=1 Win=65535 Len=0
299	4.274790200	140.116.248.21	192.168.0.100	TCP	60	80 → 64541 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
300	4.274825500	192.168.0.100	140.116.248.21	TCP	54	64541 → 80 [ACK] Seq=1 Ack=1 Win=65535 Len=0
312	4.318062000	192.168.0.100	140.116.248.21	TCP	66	65255 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
313	4.324537200	140.116.248.21	192.168.0.100	TCP	60	443 → 65255 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
314	4.324627500	192.168.0.100	140.116.248.21	TCP	54	65255 → 443 [ACK] Seq=1 Ack=1 Win=65535 Len=0
315	4.324931500	192.168.0.100	140.116.248.21	TLSv1.3	1779	Client Hello (SNI=www.ncku.edu.tw)
316	4.331610800	140.116.248.21	192.168.0.100	TCP	60	443 → 65255 [ACK] Seq=1 Ack=1726 Win=62780 Len=0
317	4.332195200	140.116.248.21	192.168.0.100	TLSv1.3	1514	Server Hello, Change Cipher Spec, Application Data
318	4.332195200	140.116.248.21	192.168.0.100	TCP	1514	443 → 65255 [ACK] Seq=1461 Ack=1726 Win=62780 Len=1460 [TCP PDU reassembled in 321]
319	4.332242500	192.168.0.100	140.116.248.21	TCP	54	65255 → 443 [ACK] Seq=1726 Ack=2921 Win=65535 Len=0
320	4.332258400	140.116.248.21	192.168.0.100	TCP	1230	443 → 65255 [PSH, ACK] Seq=2921 Ack=1726 Win=62780 Len=1176 [TCP PDU reassembled in 321]
321	4.332589800	140.116.248.21	192.168.0.100	TLSv1.3	1355	Application Data, Application Data, Application Data
322	4.332608300	192.168.0.100	140.116.248.21	TCP	54	65255 → 443 [ACK] Seq=1726 Ack=5398 Win=65535 Len=0
323	4.334553600	192.168.0.100	140.116.248.21	TLSv1.3	118	Change Cipher Spec, Application Data
324	4.334926800	192.168.0.100	140.116.248.21	TLSv1.3	146	Application Data
325	4.334984900	192.168.0.100	140.116.248.21	TLSv1.3	585	Application Data

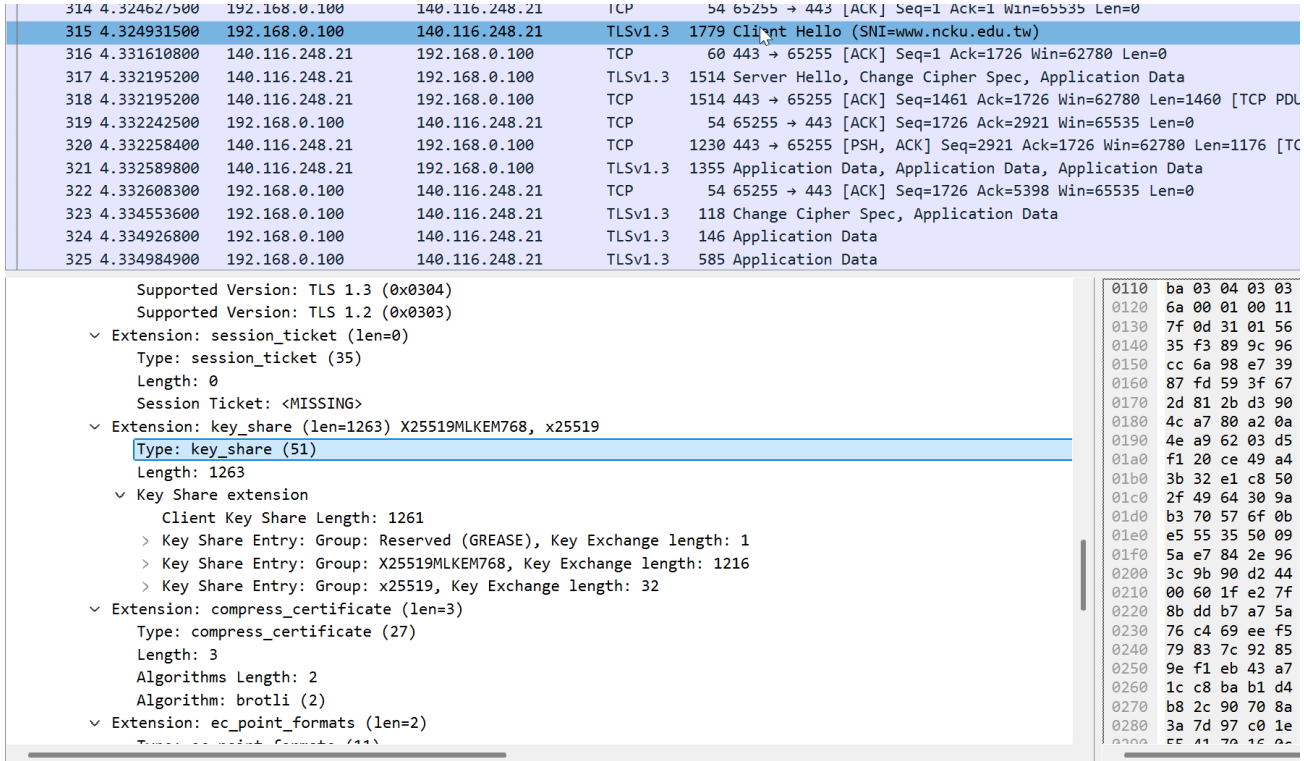
```
> Frame 315: Packet, 1779 bytes on wire (14232 bits), 1779 bytes captured (14232 bits) on interface \Device\NPF_{27DE
> Ethernet II, Src: ASUSTekCOMPU_49:31:08 (24:4b:fe:49:31:08), Dst: TPLink_02:1a:f0 (b4:b0:24:02:1a:f0)
> Internet Protocol Version 4, Src: 192.168.0.100, Dst: 140.116.248.21
> Transmission Control Protocol, Src Port: 65255, Dst Port: 443, Seq: 1, Ack: 1, Len: 1725
v Transport Layer Security
    [Stream index: 15]
  v TLSv1.3 Record Layer: Handshake Protocol: Client Hello
      Content Type: Handshake (22)
      Version: TLS 1.0 (0x0301)
      Length: 1720
    v Handshake Protocol: Client Hello
        Handshake Type: Client Hello (1)
        Length: 1716
      > Version: TLS 1.2 (0x0303)
        Random: 16ff9a76b97eff7d2dc6a62ee781075ca8d3443e427ef276307340d5d92be0bb
        Session ID Length: 32
        Session ID: e541f791145402a9ad141414e4437bd87826fee2b685b40cb74b86c7bf5ceb69
        Cipher Suites Length: 32
      > Cipher Suites (16 suites)
        Compression Methods Length: 1
```

```
0000  b4 b0 24 02 1a f0 24 4b  fe 49 31 08 08 00 45 00   ··$···$K ·I1···E·
0010  00 00 c1 9e 40 00 80 06  00 00 c0 a8 00 64 8c 74   ····@··· ·····d·t
0020  f8 15 fe e7 01 bb 4c 03  f0 7c af c8 84 c7 50 18   ····· L· ·|····P·
0030  ff ff 45 9d 00 00 16 03  01 06 b8 01 00 06 b4 03   ··E····· ····· ··
0040  03 16 ff 9a 76 b9 7e ff  7d 2d c6 a6 2e e7 81 07   ····v·~· }-····
0050  5c a8 d3 44 3e 42 7e f2  76 30 73 40 d5 d9 2b e0   \··D>B~· v0s@··+·
0060  bb 20 e5 41 f7 91 14 54  02 a9 ad 14 14 14 e4 43   ··· A··· T·····C
0070  7b d8 78 26 fe e2 b6 85  b4 0c b7 4b 86 c7 bf 5c   {·x&···· ···K···\
0080  eb 69 00 20 ea ea 13 01  13 02 13 03 c0 2b c0 2f   ·i······ ·····+·/
0090  c0 2c c0 30 cc a9 cc a8  c0 13 c0 14 00 9c 00 9d   ·,·0···· ····· ··
00a0  00 2f 00 35 01 00 06 4b  ca ca 00 00 00 05 00 05   ·/·5···K ····· ··
00b0  01 00 00 00 00 00 17 00  00 00 12 00 00 00 0a 00   ····· ·· ····· ··
00c0  0c 00 0a 6a 6a 11 ec 00  1d 00 17 00 18 00 0d 00   ···jj··· ····· ··
00d0  12 00 10 04 03 08 04 04  01 05 03 08 05 05 01 08   ····· ·· ····· ··
00e0  06 06 01 00 2d 00 02 01  01 00 00 00 14 00 12 00   ····-··· ····· ··
00f0  00 0f 77 77 77 2e 6e 63  6b 75 2e 65 64 75 2e 74   ··www.nc ku.edu.t
0100  77 44 cd 00 05 00 03 02  68 32 00 2b 00 07 06 ba   wD····· h2·+····
0110  ba 03 04 03 03 00 23 00  00 00 33 04 ef 04 ed 6a   ····· #· ··3····j
0120  6a 00 01 00 11 ec 04 c0  f3 b1 35 66 7a 28 90 8c   j····· ··5fz(··
0130  7f 0d 31 01 56 9b 2c 82  38 08 d6 e9 40 65 c5 74   ··1·V·,· 8···@e·t
0140  35 f3 89 9c 96 3f b2 20  3b 73 b5 3c 27 d2 59 da   5····?·  ;s·<'·Y·
0150  cc 6a 98 e7 39 24 51 ab  7d 95 5a 26 f8 2b e5 8b   ·j··9$Q· }·Z&·+··
0160  87 fd 59 3f 67 09 14 e9  b3 42 74 51 00 d1 e3 0b   ··Y?g··· ·BtQ····
0170  2d 81 2b d3 90 98 a2 68  7b e3 2a 07 c4 e8 9a 4c   -·+····h {·*····L
```

wireshark 乙太網路 2ENRTM3.pcapng Packets: 11708 · Displayed: 9907 (84.6%) · Dropped: 0 (0.0%) Profile: Default

SSL key exchange

- Diffie-Hellman key exchange
- RSA-based key exchange
  - Encrypt secret *s* with public key of server
  - (resists attacks described in last class)



Reaction attack

- Send encrypted nonce, see if it is of the right form
  - If padding decrypts incorrectly, server sends an error
  - Otherwise, SSL protocol continues
- Use algebraic properties of RSA to deduce bits of key from error
- Fixed with better padding
- Better fix: don’t send error
  - Pretend key exchange was OK, continue protocol with bogus session key

SSL authentication

- Anonymous (no authentication)
- RSA authentication (implicit)
- Sign Diffie-Hellman parameters
  - Achieves *perfect forward secrecy*

SSL session

- Use ChangeCipherSpec message to start encrypting data
- Encryption: RC4, also DES, 3DES, AES, ...
- Authentication: HMAC, using MD5 or SHA1

```
> Internet Protocol Version 4, Src: 140.113.42.195, Dst: 192.168.0.100
> Transmission Control Protocol, Src Port: 443, Dst Port: 52555, Seq: 431854, Ack: 16927, Len: 1460
> [5 Reassembled TCP Segments (7300 bytes): #20821(1460), #20823(1460), #20824(1460), #20826(1460), #20827(1460)]
v Transport Layer Security
    [Stream index: 79]
  v TLSv1.2 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
      Content Type: Application Data (23)
      Version: TLS 1.2 (0x0303)
      Length: 7295
      Encrypted Application Data […]: 4000000000000000884ebf06279fbc4d75abd94734b9a2fa511454c1eff0b6f74269c006d444c5b5
      [Application Data Protocol: Hypertext Transfer Protocol]
```

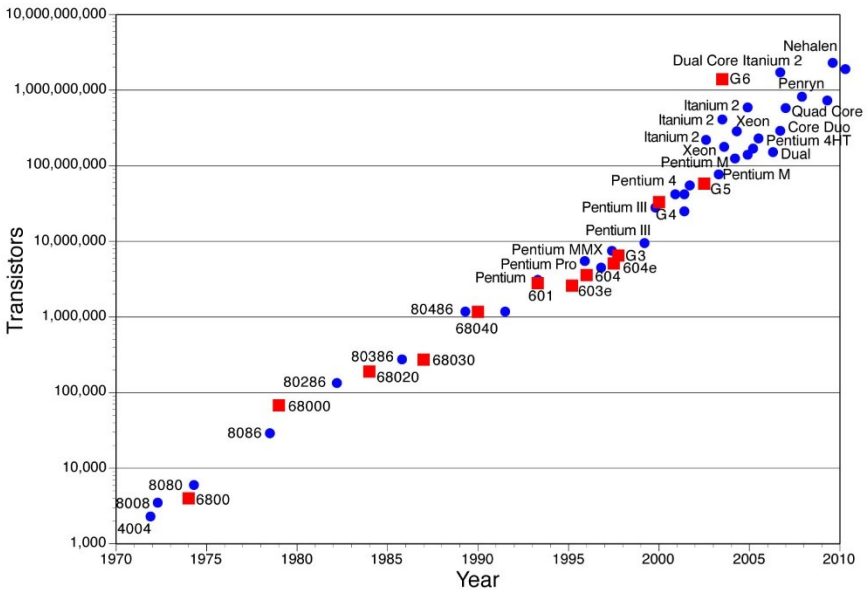
```
0020  c1 ef f0 b6 f7 42 69 c0  06 d4 44 c5 b5 3d 20 38  ·····Bi· ··D··= 8
0030  47 92 d3 02 e7 1b 2b 9c  dd 3e a2 fe eb 2b 94 63  G·····+· ·>···+·c
0040  90 7d bd 6a b8 c8 20 d7  39 b5 77 af c3 67 31 06  ·}·j·· · 9·w··g1·
0050  ca 2a 0f 5e a8 b1 96 1a  71 82 0f 08 5d 50 c3 81  ·*·^···· q···]P··
0060  7f 04 f3 b6 5e ce ff 22  c2 f9 8d 1e 36 16 c9 5b  ····^··" ····6··[
0070  d7 e9 c8 c4 f6 8f c9 46  e2 a2 14 eb 6a 3f 29 07  ·******F ····j?)·
0080  f4 7e 09 84 20 ae 32 a6  37 49 c4 ac f3 e5 2c 49  ·~·· ·2· 7I····,I
0090  b0 40 75 68 92 7d b7 68  45 2f 18 09 20 ff ff 50  ·@uh·}·h E/·· ··P
00a0  71 f3 d1 24 b2 ec 22 62  dc 0d 6c 26 db 62 83 2f  q··$··"b ··l&·b·/
00b0  c9 be e5 99 c9 a8 21 66  f4 fe e8 6f 27 5f 34 51  ******!f ···o'_4Q
00c0  d7 d3 e9 04 89 32 3d cc  f1 a8 c0 f7 ac ed 92 ce  *****2=· ********
00d0  be bc 99 6f 7a 6c d2 a0  66 5d 08 37 ef 37 15 89  ···ozl·· f]·7·7··
00e0  68 a0 c6 9f fc fd 7d f5  06 28 83 07 5f fe f1 07  h*****}· ·(··_···
00f0  53 fd a9 f8 4c fb 0d 2d  26 c2 ea c7 32 aa 74 3e  S···L··· &···2·t>
0100  d8 1a 62 b6 bf 0c af 48  a2 c9 8c ab c1 98 35 5e  ··b····H ******5^
0110  5d 13 bb 8e 47 36 d3 68  79 4d c1 6b c6 22 b9 d6  ]···G6·h yM·k·"··
0120  19 47 c6 5c 51 83 d6 e9  27 ac 50 d0 fa 6e f0 61  ·G·\Q··· '·P··n·a
0130  3f 71 9d 10 a5 14 dc 1f  ba a5 75 1f 4a 58 ad 75  ?q******  ··u·JX·u
0140  f3 3b 4f ee 80 53 75 35  8a 72 7c d4 20 89 6a 26  ·;O··Su5 ·r|· ·j&
0150  dd 79 0c 41 6c 34 c2 12  14 26 85 06 dd 79 f3 d9  ·y·Al4·· ·&···y··
```

SSL pitfalls

- Hard to set up
  - Expensive certificates
  - Resource-intensive
- Insufficient verification
  - Do people notice the lock icon?
  - Do people check the URL?
- Improper use

Forward Secrecy

- Under traditional HTTPS
  - The client chooses a random session key
  - Encrypt it using the server's RSA public key
  - Send **the encrypted session key over the network** to the server.
  - The server decrypts the session key with its RSA private key and use the decrypted session key to communicate with the client
- What if an adversary records all the encrypted traffic
  - And eventually, he breaks the server's RSA private key…



Forward Secrecy

- A session key derived from a set of long-term keys cannot be compromised if one of the long-term keys is compromised in the future
- How to achieve this?
  - Use Diffie-Hellman to establish session key
    - Session key is not transmitted on the network
  - Server's private key is only used to sign (authenticate) the key exchange to prevent man-in-the-middle attacks
    - Note that Diffie-Hellman works only for **authenticated** non-private channel.

Secure Shell

- SSH: secure shell
- Designed in 1995 by Tatu Ylonen
- Replaced in 1996 by SSHv2
  - Fixed security holes
  - Eventually standardized
- Less popular than SSL
  - But completely dominates the market

SSH architecture

- Similar to SSL:
  - Clients, servers, socket-like interface
- Difference:
  - No certificates
  - Remember public key associated with host
- Intuition
  - MITM attacks are difficult
  - To evade detection, adversary must succeed at MITM *every time*

Other SSH features

- Flexible authentication architecture
  - Password, public key, SecureID, Kerberos, ...
- Perfect forward secrecy
  - Permanent host key + temporary key
  - Host key signs temporary key
  - Key exchange done with temporary key
- Port forwarding
  - Used to tunnel traffic on other ports
  - Esp. X11 forwarding

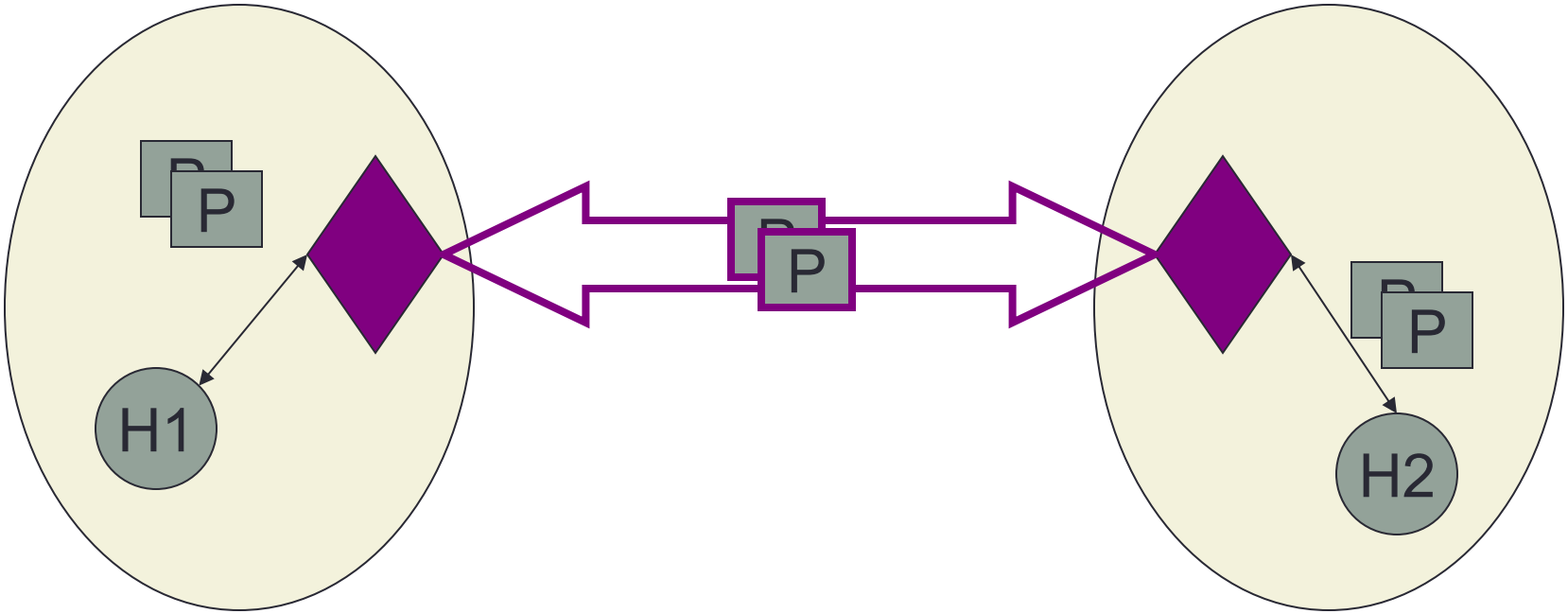
IPsec

- Designed as part of IPv6 suite
  - One of the key features v6 was supposed to bring
- Backported to IPv4
- Two options: AH (authentication) and ESP (encapsulated security)
- Two modes: transport and tunnel

Transport vs. Tunnel Mode

- Grand vision: eventually, all IP packets will be encrypted and authenticated
- Transport mode: add headers to IP to do so
- Reality: Most computers don’t support IPsec
- Tunnel mode: use IPsec between two gateways to relay IP packets through “untrusted cloud”

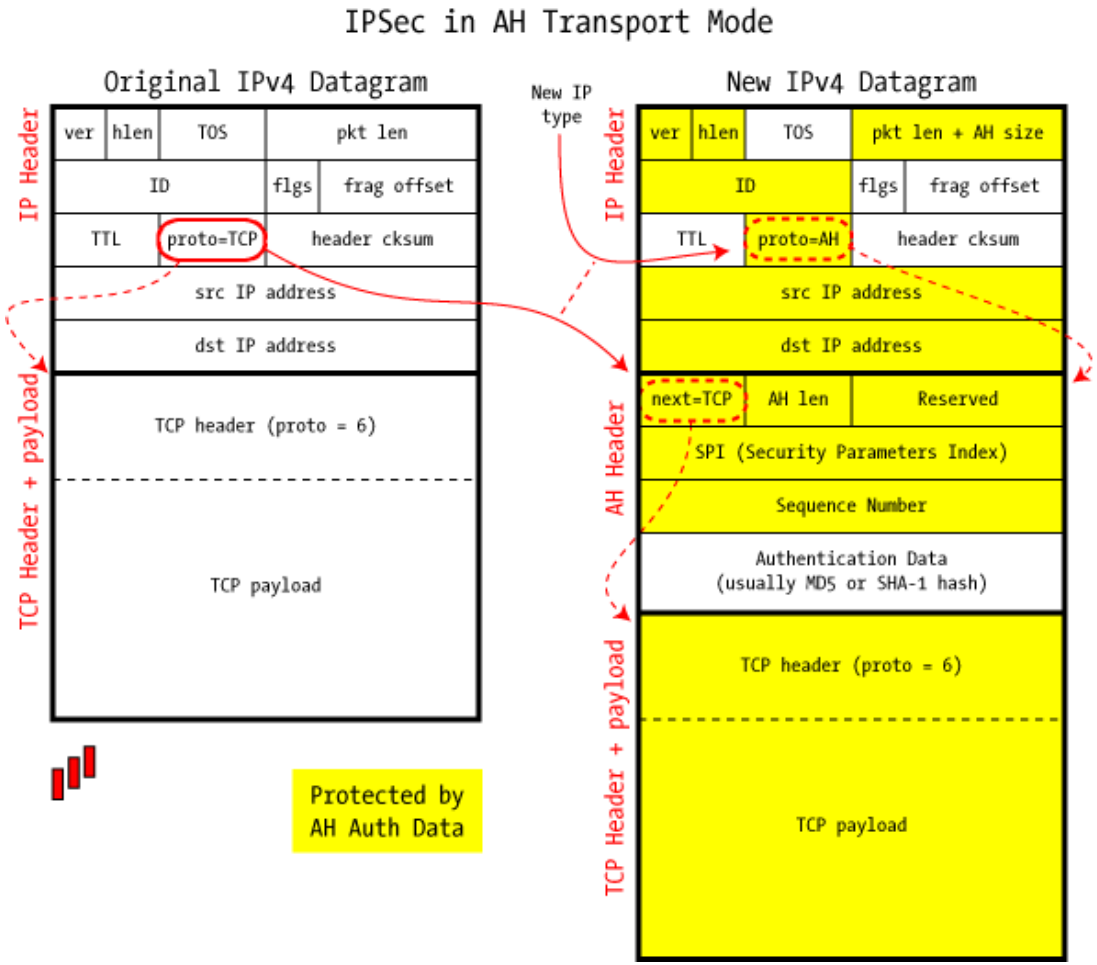
Tunnel Mode



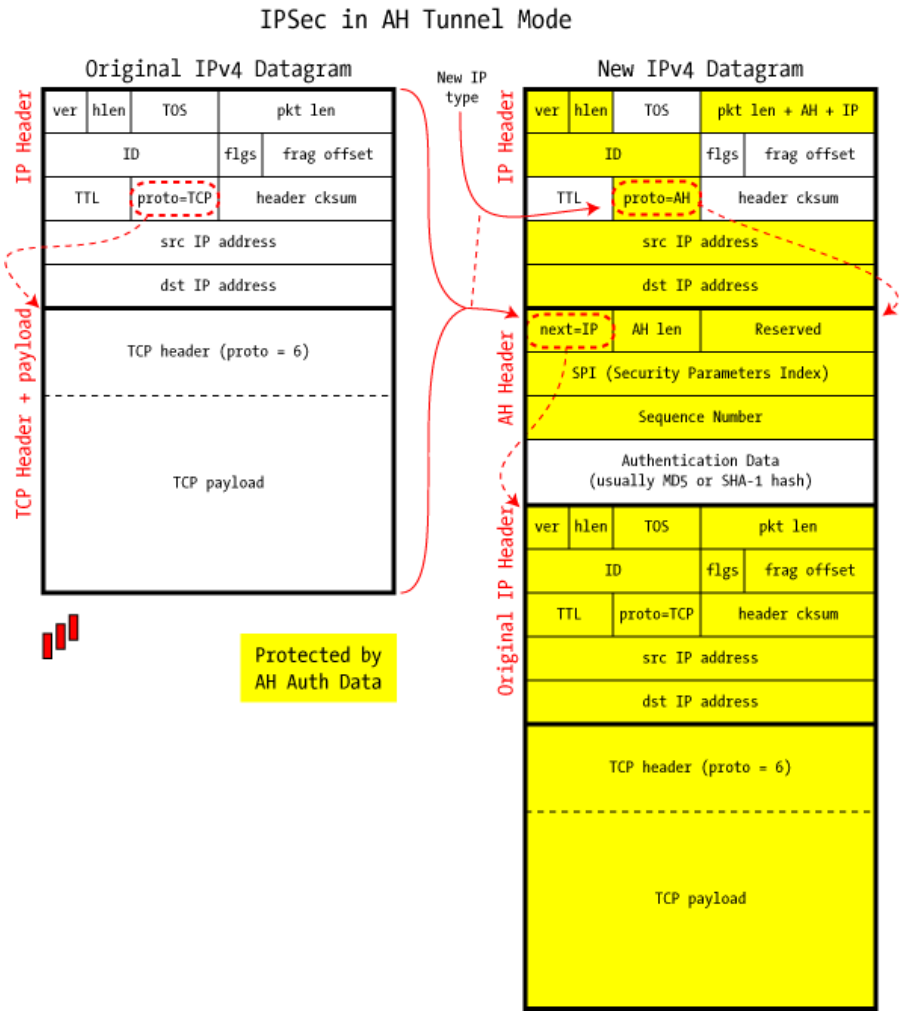
AH - Authentication

- Simple design: add header with authentication data
  - Security parameters
  - Authentication data (usu. SHA1-HMAC)

AH diagram


IPSec in AH Transport Mode
Original IPv4 Datagram
New IPv4 Datagram
New IP type
IP Header
ver
hlen
TOS
pkt len
ID
flgs
frag offset
TTL
proto=TCP
header cksum
src IP address
dst IP address
TCP Header + payload
TCP header (proto = 6)
TCP payload
Protected by AH Auth Data
IP Header
ver
hlen
TOS
pkt len + AH size
ID
flgs
frag offset
TTL
proto=AH
header cksum
src IP address
dst IP address
AH Header
next=TCP
AH len
Reserved
SPI (Security Parameters Index)
Sequence Number
Authentication Data
(usually MD5 or SHA-1 hash)
TCP Header + payload
TCP header (proto = 6)
TCP payload

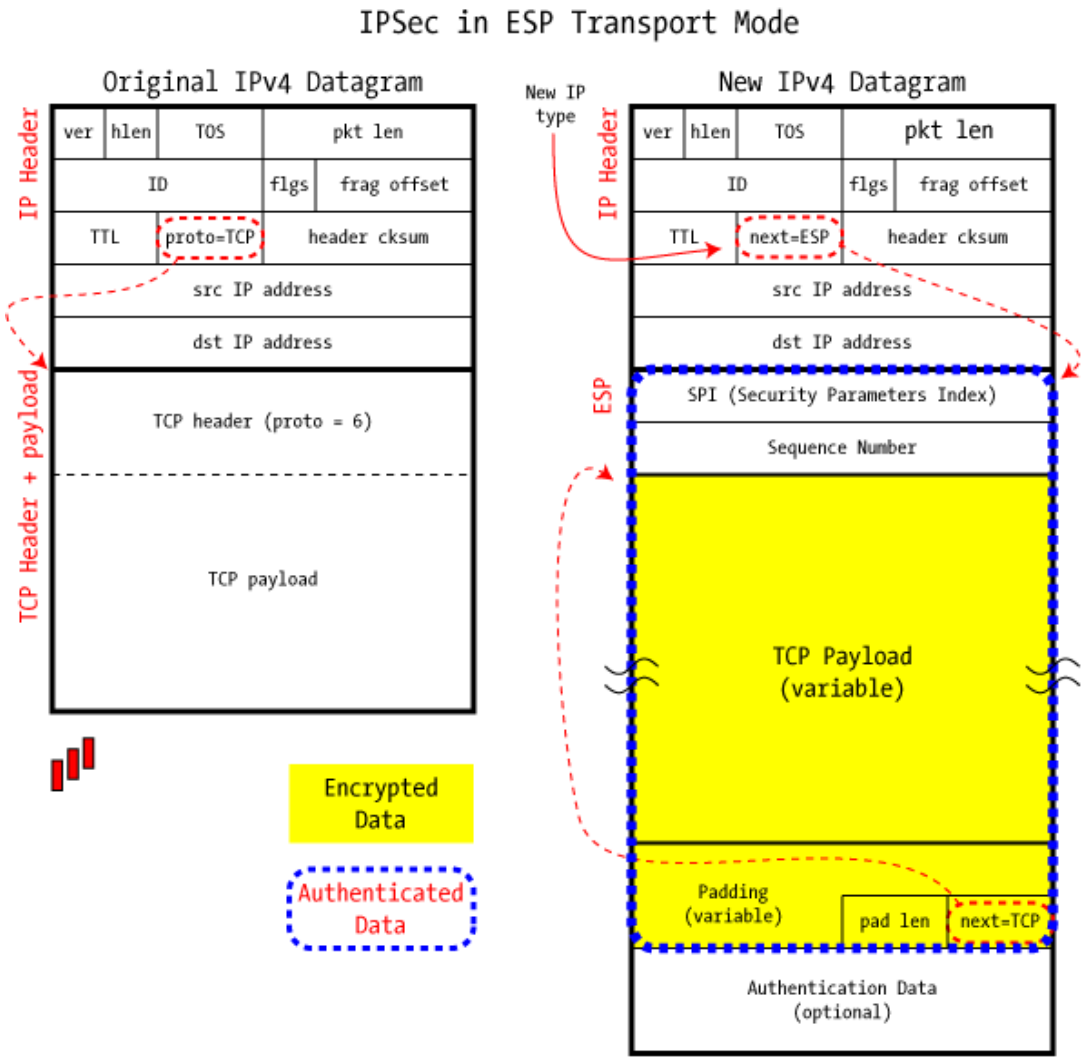
AH in Tunnel Mode



ESP - Encapsulated Security Payload

- Encapsulate data
  - Encapsulate datagram rather than add a header
  - Encrypt & authenticate

ESP diagram


IPSec in ESP Transport Mode
Original IPv4 Datagram
IP Header
ver
hlen
TOS
pkt len
ID
flgs
frag offset
TTL
proto=TCP
header cksum
src IP address
dst IP address
TCP Header + payload
TCP header (proto = 6)
TCP payload
New IP type
New IPv4 Datagram
IP Header
ver
hlen
TOS
pkt len
ID
flgs
frag offset
TTL
next=ESP
header cksum
src IP address
dst IP address
ESP
SPI (Security Parameters Index)
Sequence Number
TCP Payload (variable)
Padding (variable)
pad len
next=TCP
Authentication Data (optional)
Encrypted Data
Authenticated Data

Key management

- ESP and AH use *session keys*
- Sessions are called *Security Associations*
  - Indexed by protocol, IP address, SPI
- ISAKMP: Internet Security Association Key Management Protocol
  - Authenticates parties
  - Establishes session keys
- Authentication
  - Big global PKI (DNSSEC??)
  - Manual configuration

IPsec redux

- Deployment of IPsec limited
- Some reasons
  - Global PKI infrastructure hard to set up
  - Wrong layer for security
    - Session/Application-layer, rather than Network layer
  - Fixes a “solved” problem
    - SSL & SSH work well
- IPsec success: VPNs
  - Use tunnel mode of Ipsec

PKI

憑證管理員

您的憑證 | 人員 | 伺服器 | 憑證機構 | 其他

您的憑證可識別下列憑證機構:

憑證名稱	安全裝置
GlobalSign nv-sa	
AlphaSSL CA - G2	軟體安全裝置
GlobalSign	軟體安全裝置
GlobalSign Organization Validation CA	軟體安全裝置
GlobalSign Root CA	Builtin Object Token
GlobalSign Organization Validation CA - G2	軟體安全裝置
GlobalSign Domain Validation CA - G2	軟體安全裝置
GlobalSign Organization Validation CA - SHA256 - G2	軟體安全裝置
GlobalSign Organization Validation CA - G2	軟體安全裝置
AlphaSSL CA - SHA256 - G2	軟體安全裝置
GlobalSign Domain Validation CA - G2	軟體安全裝置
GlobalSign Domain Validation CA - SHA256 - G2	軟體安全裝置
GoDaddy.com, Inc.	
Go Daddy Root Certificate Authority - G2	Builtin Object Token
Go Daddy Secure Certificate Authority - G2	軟體安全裝置
Government Root Certification Authority	
Taiwan GRCA	Builtin Object Token
Government Root Certification Authority	軟體安全裝置
政府憑證管理中心	軟體安全裝置
政府憑證管理中心	軟體安全裝置
GTE Corporation	
Microsoft Internet Authority	軟體安全裝置

檢視 (V)... | 編輯信任 (E)... | 匯入 (M)... | 匯出 (X)... | 刪除或取消信任 (D)...

確定

Recent problems with SSL

- Issues with the security of Certificate Authorities
  - Comodo, Diginotar, KPN, Trustwave, …
- News on international espionage
  - Attacks against CAs
  - Compelled certificate attack (i.e. a government orders a CA to issue a false certificate)

Recent problems with SSL

- Weaknesses in the protocol
  - renegotiation, BEAST, CRIME, etc.
- Weaknesses in SSL implementations
  - gotofail, heartbleed, change cipher spec (CCS) injection, etc.
- Weak SSL keys in large numbers (0.2% of all keys on the web)

Issues of Certificate Authorities

- There are about 100+ CAs globally; Browsers trust all of them
  - If one CA is compromised, the attacker can impersonate any website
- Just having a few CAs is problematic
  - CAs operate in different countries and jurisdictions
    - They don't really trust each other
  - May work for certain applications
    - E.g. LINE,…
- Self-signed certificates
  - Man-in-the-middle attacks
- Self-signed certificates over a secure channel
  - Does not scale

Trust on First Use

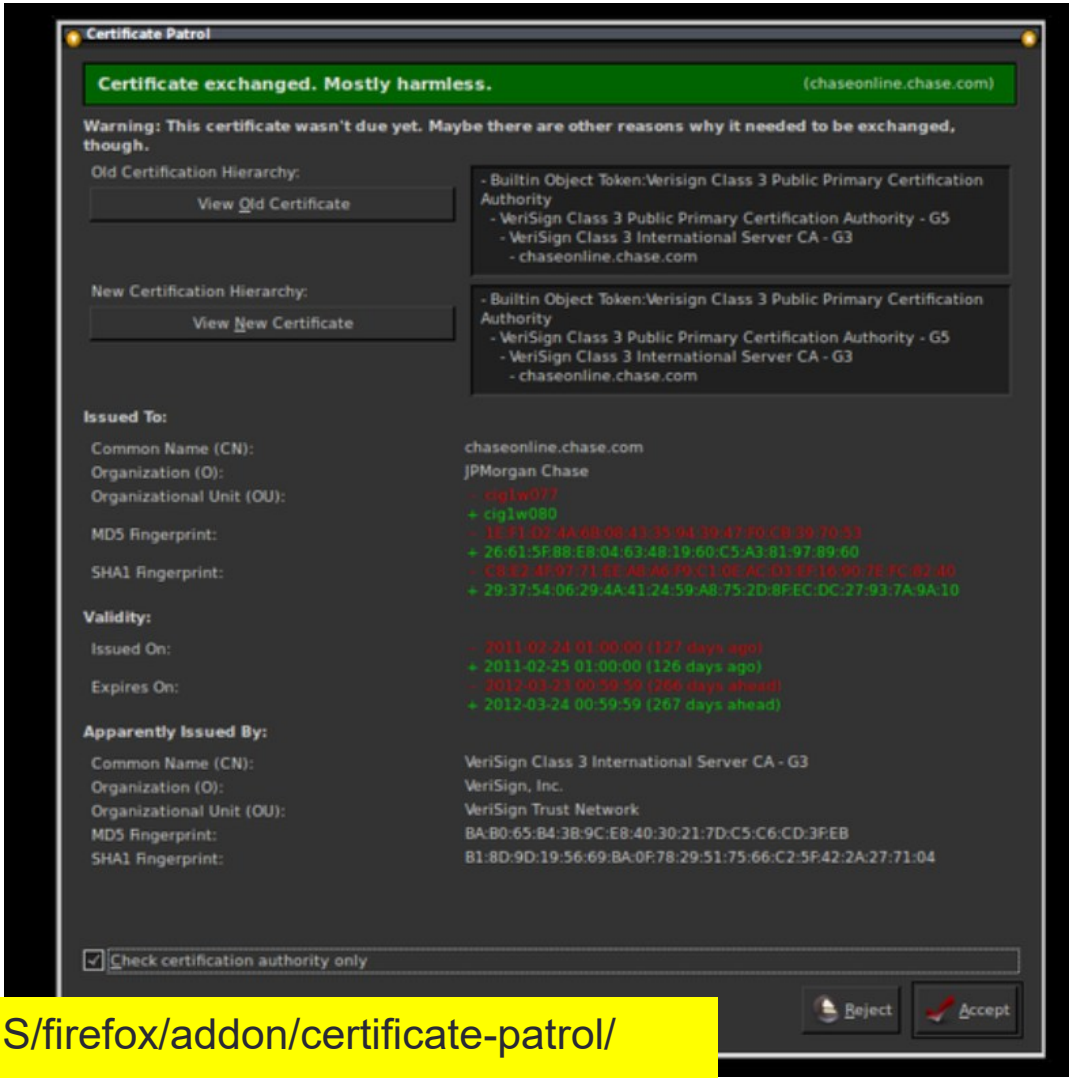
linux1.cs.nctu.edu.tw - PuTTY

```
linux1 [/u/faculty/ysw] -ysw- % ssh dsns.cs.nctu.edu.tw
The authenticity of host 'dsns.cs.nctu.edu.tw (140.113.210.234)' can't be established.
RSA key fingerprint is 6b:11:fd:e6:c3:51:4c:16:0f:c7:b8:63:ed:fc:42:9e.
Are you sure you want to continue connecting (yes/no)?
```

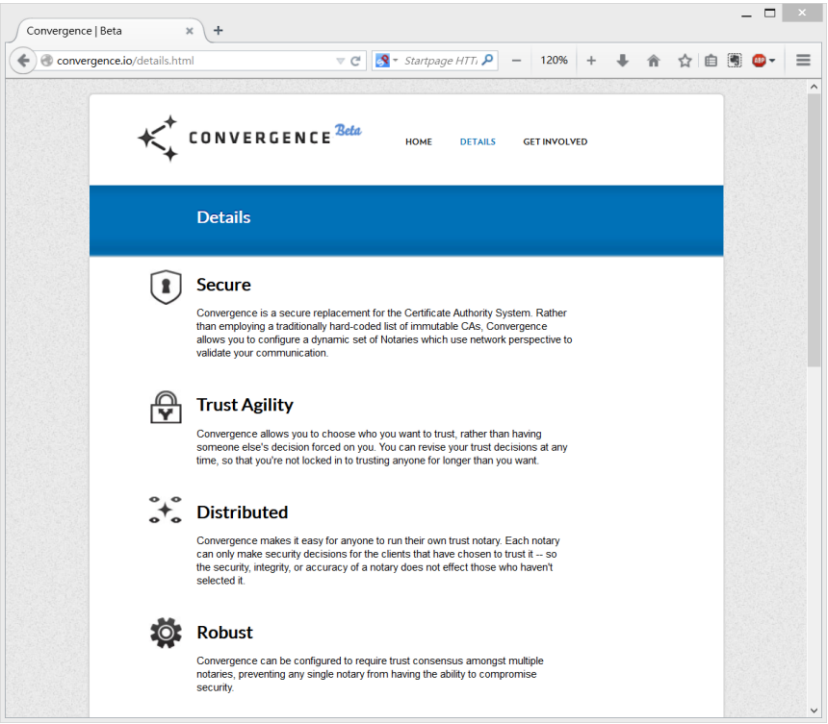
Certificate Patrol

- Implement certificate pinning

- Same certificate?
- Same key?
- Same CA?



- https://addons.mozilla.org/en-US/firefox/addon/certificate-patrol/

Networked verification of identify

- Assumption: attacker may attack a single client but not the whole web
- http://convergence.io/